

Capstone Project Report

Student Name: Shreya Roy

Project Type: Capstone Project

Target Application: TutorialsNinja Demo Store

Capstone Project Report: Enterprise-Grade E-Commerce Automated Testing Framework

1. Project Overview & Motivation

When planning my capstone project, I wanted to build something that reflects real-world software testing standards. Manual testing doesn't scale well for large e-commerce platforms, so I decided to develop an automated end-to-end testing framework from scratch.

For this project, I chose the **TutorialsNinja** demo application as my target store. The goal was to automate critical user journeys—such as user registration, login authentication, product searching, and cart management—while ensuring the code is modular, reusable, and easy to maintain.

2. Engineering Approach & Architecture

To keep the framework clean and industry-compliant, I avoided writing messy, hardcoded test scripts. Instead, I structured the project around two core design patterns:

- **Page Object Model (POM):** I separated the test scripts completely from the web element locators and page actions. Every web page (like the login screen or search results) has its own dedicated class inside `source_pages/`. If a button ID changes on the website, I only have to update it in one place.
- **Data-Driven Testing (DDT):** Rather than hardcoding test inputs directly into the code, I externalized test data using `.csv` files stored in `csv_data/`. This allows the test suite to run dynamically across multiple sets of user inputs and search queries.
- **DRY Principle & Explicit Waits:** Using a base action class (`BaseActions`), I centralized common Selenium commands and integrated robust explicit waits to prevent flaky tests and synchronization issues.

3. Project Directory Structure:

Here is how the framework is organized within Folder 2 of my repository:

Selenium_python_framework_development/

```
|
|
|— configs/          # Environment settings and base configurations
|— csv_data/         # External test input data (test_inputs.csv)
|— docs/             # Project report and documentation (this file)
|— failure_screenshots/ # Captured images of test failures and debugging logs
|— html_reports/     # Generated execution_report.html dashboards
|— source_pages/     # Page Object Model classes (BaseActions, LoginScreen, SearchScreen)
|— test_scripts/     # End-to-end test execution suites
|— utils/            # Helper utilities and data parsers
|— report_capstone_project.pdf #Comprehensive document detailing the test results
|— conftest.py       # PyTest fixtures, hooks, and configuration setup
|— requirements.txt   # Python package dependencies
|— README.md         # Framework level documentation
```

4. Execution Workflow & Setup

The framework runs on Python 3 and uses pytest as the test runner, integrated with pytest-html for executive reporting.

1. Install Dependencies:

```
pip install -r requirements.txt
```

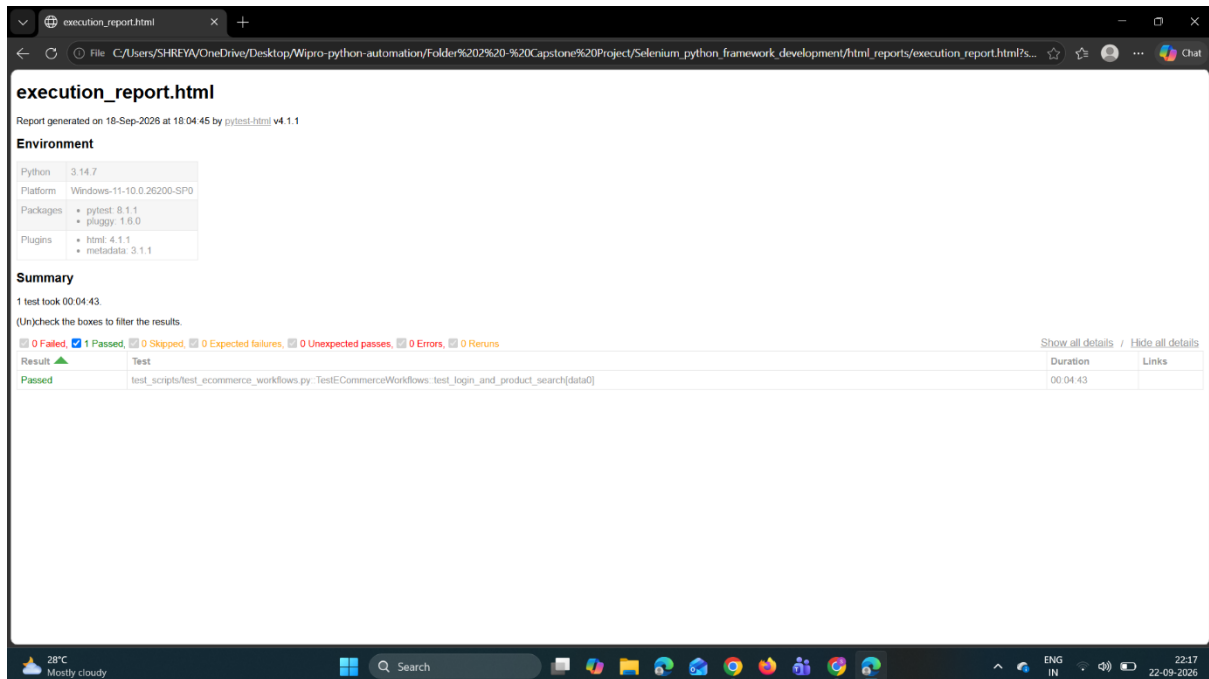
2. Run Tests & Generate Report:

```
pytest --html=html_reports/execution_report.html --self-contained-html
```

5. Results, Outputs, & Screenshots

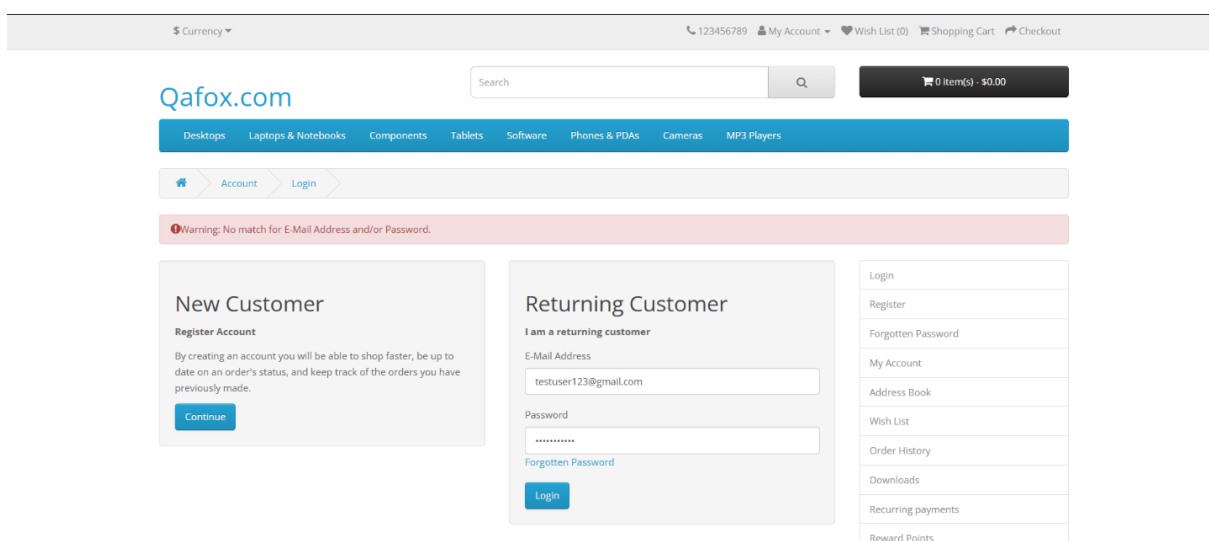
Test Execution & Reporting

The test suite executes cleanly, parsing data from the CSV files and validating user flows on TutorialsNinja. Once finished, it compiles an interactive HTML dashboard.



Failure Handling & Debugging

To make troubleshooting easier, I implemented failure hooks in conftest.py. If a test assertion fails, the framework automatically snaps a screenshot of the browser state at that exact moment.



[Rejected the credentials (testuser123@gmail.com) because that account isn't registered in the demo store.]

6. Conclusion

Building this framework gave me hands-on experience solving real QA automation challenges—handling element synchronization, structuring large test suites with the Page Object Model, and generating professional reports. It successfully fulfills all the requirements for my capstone submission.